



本小节内容

二叉树层次建树实战

二叉树层次建树实战

在上一小节我们已经二叉树的原理，对层次建树的流程进行了解析。

为了提高代码的编写效率，我们把结构体类型的声明放入 `function.h` 头文件，`function.h` 头文件在 `main.cpp` 进行了 `include`，最终建树的对不对，我们可以通过单步调试来看树的结果，断点打在 `return 0` 位置，下面直接上代码：

`function.h` 代码如下

```
#include <stdio.h>
#include <stdlib.h>

typedef char BiElemType;
typedef struct BiTNode{
    BiElemType c;//c 就是书籍上的 data
    struct BiTNode *lchild;
    struct BiTNode *rchild;
}BiTNode,*BiTree;

//tag 结构体是辅助队列使用的
typedef struct tag{
    BiTree p;//树的某一个结点的地址值
    struct tag *pNext;
}tag_t,*ptag_t;
```

下面是 `main.cpp` 的代码

```
#include "function.h"

//《王道 C 督学营》课程
//二叉树的建树（层次建树）
int main()
{
    BiTree pnew;//用来指向新申请的树结点
    char c;
    BiTree tree=NULL;//树根
```



```
//phead 就是队列头，ptail 就是队列尾
ptag_t phead=NULL,ptail=NULL,listpnew=NULL,pcur=NULL;
//输入内容为 abcdefghij
while(scanf("%c",&c))
{
    if(c=='\n')
    {
        break;
    }
    pnew=(BiTree)calloc(1,sizeof(BiTNode));//calloc 申请空间并对空间进行初始化，赋值为 0
    pnew->c=c;//数据放进去
    listpnew=(ptag_t)calloc(1,sizeof(tag_t));//给队列结点申请空间
    listpnew->p=pnew;
    if(NULL==tree)
    {
        tree=pnew;//树的根
        phead=listpnew;//队列头
        ptail=listpnew;//队列尾
        pcur=listpnew;
        continue;
    }else{
        ptail->pnext=listpnew;//新结点放入链表，通过尾插法
        ptail=listpnew;//ptail 指向队列尾部
    }//pcur 始终指向要插入的结点的位置
    if(NULL==pcur->p->lchild)//如何把新结点放入树
    {
        pcur->p->lchild=pnew;//把新结点放到要插入结点的左边
    }else if(NULL==pcur->p->rchild)
    {
        pcur->p->rchild=pnew;//把新结点放到要插入结点的右边
        pcur=pcur->pnext;//左右都放了结点后，pcur 指向队列的下一个
    }
}
return 0;
}
```